

Demand Forecasting & Inventory Optimization

Goal: Answer two business questions for the top-selling product (ID 365):

1. What is the demand forecast for the next 2 years?
2. What is the optimal inventory level?

Results:

- **Q1 - 2-Year Forecast:** 3.17M consensus (Prophet 3.07M, RF 3.23M, XGB 3.21M) - 2.1% variation
- **Q2 - Inventory Policy:** Reorder at 1,067 units, order 392 units (EOQ), maintain 575 units safety stock
- **Model Accuracy:** All models ~19.5% MAPE (XGBoost best at 19.4%, Prophet/RF at 19.5%)

In [495...

```
import pandas as pd
import numpy as np
import math
import matplotlib.pyplot as plt
import seaborn as sns
import squarify
import warnings, logging, sys, io

from sklearn.model_selection import ParameterGrid
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error
import xgboost as xgb
import optuna
from prophet import Prophet
from prophet.diagnostics import cross_validation, performance_metrics

# Suppress all output noise
warnings.filterwarnings('ignore')
for logger in ['cmdstanpy', 'prophet', 'cmdstan']:
    logging.getLogger(logger).disabled = True
optuna.logging.set_verbosity(optuna.logging.WARNING)

class SuppressStderr:
    def __enter__(self): self._stderr, sys.stderr = sys.stderr, io.StringIO()
    def __exit__(self, *args): sys.stderr = self._stderr
```

1. Load Data

In [455...

```
# Load dataset and keep relevant columns
cols = ['order date (DateOrders)', 'Sales', 'Sales per customer', 'Order Product Card Id', 'Product Price', 'Delivery Status', 'Market', 'Customer Country', 'Shipping Mode', 'Type']

df = pd.read_csv('DataCoSupplyChainDataset.csv', encoding='ISO-8859-1')[cols]
```

```
df['order date (DateOrders)'] = pd.to_datetime(df['order date (DateOrders)'])

# Extract time features
df['Month'] = df['order date (DateOrders)'].dt.month
df['Year'] = df['order date (DateOrders)'].dt.year
df['Day'] = df['order date (DateOrders)'].dt.day
df = df.drop(columns=['order date (DateOrders)'])

print(f"Loaded {len(df):,} transactions | {len(df.columns)} columns")
print(f"Date range: {df['Year'].min()}--{df['Month'].min():02d} to {df['Year'].max()}")
print(f"Missing values: {df.isnull().sum().sum()}")
```

Loaded 180,519 transactions | 14 columns
Date range: 2015-01 to 2018-12
Missing values: 0

```
In [456... # Identify top-selling product
top_product_id = df['Product Card Id'].value_counts().index[0]
top_product = df[df['Product Card Id'] == top_product_id].copy()

print(f"Top product: ID {top_product_id}")
print(f"  Orders: {len(top_product):,} ({len(top_product)/len(df)*100:.1f}% of all transactions)")
print(f"  Price: ${top_product['Product Price'].iloc[0]:.2f}")
print(f"  Total Sales: ${top_product['Sales'].sum():,.0f}")
```

Top product: ID 365
Orders: 24,515 (13.6% of all transactions)
Price: \$59.99
Total Sales: \$4,421,143

2. Exploratory Data Analysis

```
In [457... # Distribution plots
fig, axes = plt.subplots(2, 3, figsize=(14, 8))

sns.countplot(data=df, x='Product Card Id', ax=axes[0,0],
              order=df['Product Card Id'].value_counts().iloc[:10].index)
axes[0,0].set_title('Top 10 Products')

sns.histplot(data=df, x='Sales', kde=True, ax=axes[0,1])
axes[0,1].set_title('Sales Distribution')

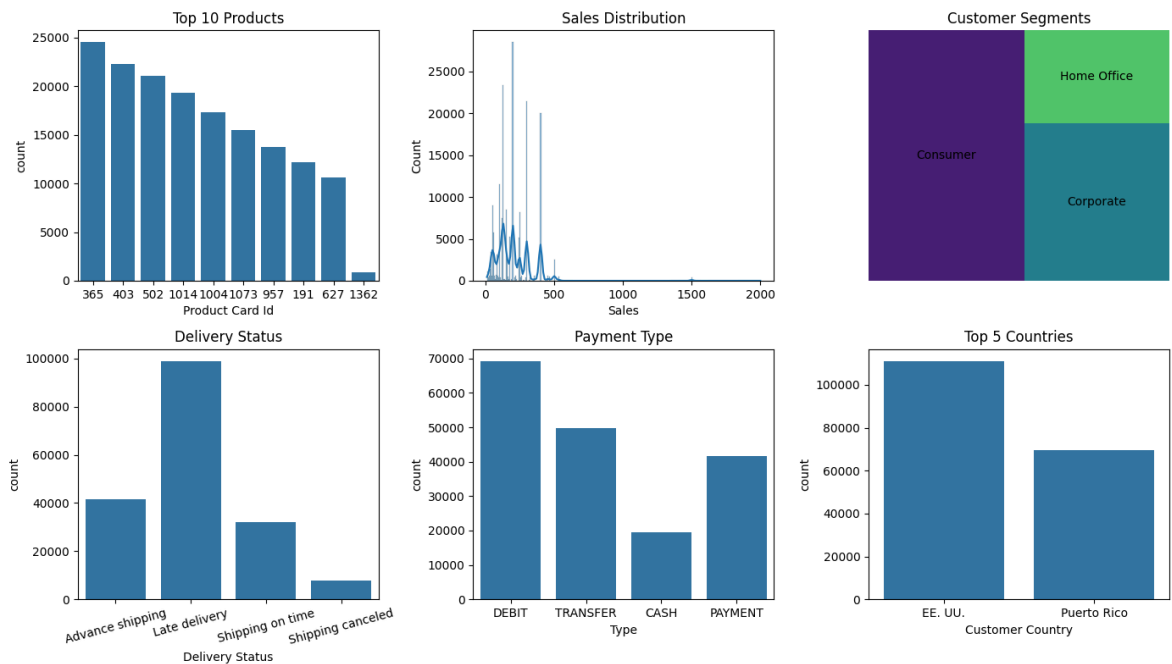
squarify.plot(sizes=df['Customer Segment'].value_counts(),
              label=df['Customer Segment'].value_counts().index, ax=axes[0,2])
axes[0,2].set_title('Customer Segments')
axes[0,2].axis('off')

sns.countplot(data=df, x='Delivery Status', ax=axes[1,0])
axes[1,0].set_title('Delivery Status')
axes[1,0].tick_params(axis='x', rotation=15)

sns.countplot(data=df, x='Type', ax=axes[1,1])
axes[1,1].set_title('Payment Type')

sns.countplot(data=df, x='Customer Country', ax=axes[1,2],
              order=df['Customer Country'].value_counts().iloc[:5].index)
axes[1,2].set_title('Top 5 Countries')
```

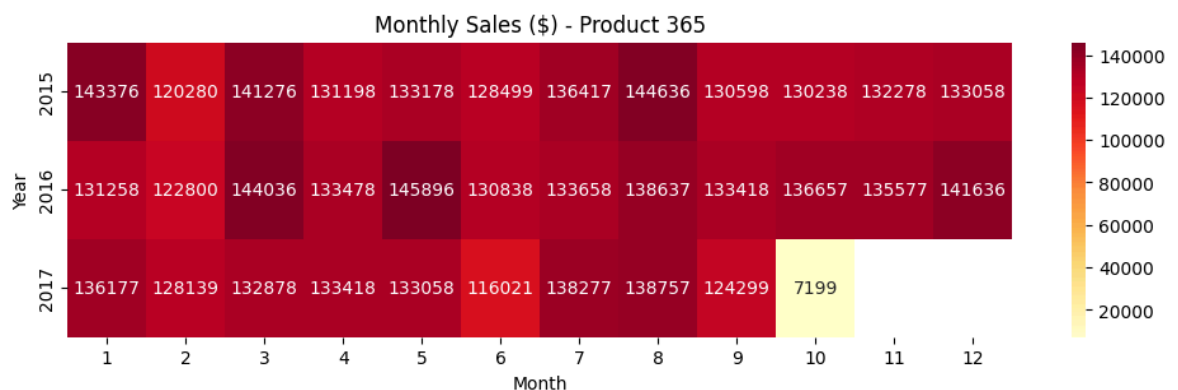
```
plt.tight_layout()
plt.show()
```



EDA Observations:

- Product 365 is the top seller with 24,515 orders (13.6% of all transactions) and \$4.4M total revenue
- Sales distribution is right-skewed: most transactions are small, with fewer high-value orders
- Consumer segment dominates purchases; Corporate and Home Office are secondary
- 55% late delivery rate is a significant operational issue that could impact customer retention
- Europe and LATAM are the strongest markets for this product
- Monthly sales average 130K but vary widely (7K to \$146K), indicating seasonality

```
In [458... # Monthly sales heatmap for Product 365
pivot = top_product.pivot_table(values='Sales', index='Year', columns='Mo
plt.figure(figsize=(12, 3))
sns.heatmap(pivot, annot=True, fmt='.0f', cmap='YlOrRd')
plt.title(f'Monthly Sales ($) - Product {top_product_id}')
plt.show()
```



```

In [459... # Monthly sales trend for Product 365
monthly_sales = top_product.groupby(['Year', 'Month'])['Sales'].agg(['sum'])
monthly_sales['Date'] = pd.to_datetime(monthly_sales[['Year', 'Month']].a

fig, axes = plt.subplots(1, 2, figsize=(14, 4))

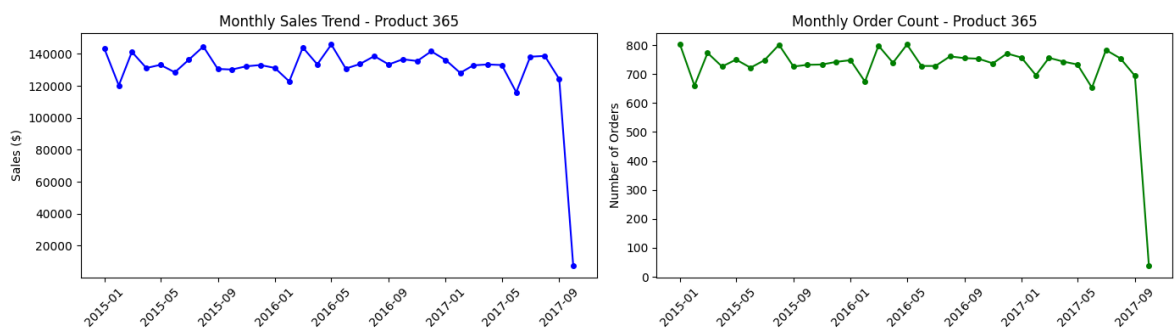
# Sales trend
axes[0].plot(monthly_sales['Date'], monthly_sales['sum'], 'b-o', markersize=10)
axes[0].set_title(f'Monthly Sales Trend - Product {top_product_id}')
axes[0].set_ylabel('Sales ($)')
axes[0].tick_params(axis='x', rotation=45)

# Order count trend
axes[1].plot(monthly_sales['Date'], monthly_sales['count'], 'g-o', markersize=10)
axes[1].set_title(f'Monthly Order Count - Product {top_product_id}')
axes[1].set_ylabel('Number of Orders')
axes[1].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()

print(f"Average monthly sales: ${monthly_sales['sum'].mean():,.0f}")
print(f"Sales range: ${monthly_sales['sum'].min():,.0f} - ${monthly_sales['sum'].max():,.0f}")

```



Average monthly sales: \$130,034
Sales range: \$7,199 – \$145,896

```

In [460... # Market and delivery analysis for Product 365
fig, axes = plt.subplots(1, 3, figsize=(14, 4))

# Market distribution
market_sales = top_product.groupby('Market')['Sales'].sum().sort_values(ascending=False)
market_sales.plot(kind='barh', ax=axes[0], color='steelblue')
axes[0].set_title('Sales by Market')
axes[0].set_xlabel('Total Sales ($)')

# Delivery status
delivery = top_product['Delivery Status'].value_counts()
colors = ['#ff6b6b' if 'Late' in x else '#4ecdc4' for x in delivery.index]
delivery.plot(kind='bar', ax=axes[1], color=colors)
axes[1].set_title('Delivery Status')
axes[1].set_ylabel('Count')
axes[1].tick_params(axis='x', rotation=45)

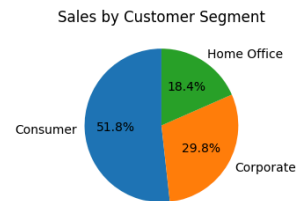
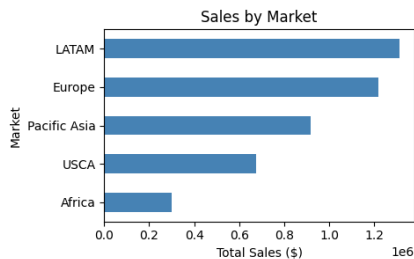
# Customer segment
segment_sales = top_product.groupby('Customer Segment')['Sales'].sum()
axes[2].pie(segment_sales, labels=segment_sales.index, autopct='%1.1f%%', shadow=True)
axes[2].set_title('Sales by Customer Segment')

plt.tight_layout()

```

```
plt.show()
```

```
late_pct = top_product[top_product['Delivery Status'].str.contains('Late')]
print(f"Late delivery rate: {late_pct:.1f}%")
```



Late delivery rate: 55.0%

3. Prophet Forecasting

Forecast daily sales for Product 365 using Facebook Prophet.

```
In [461... # Prepare data for Prophet: aggregate daily sales
prophet_df = (
    top_product
    .assign(ds=lambda x: pd.to_datetime(x[['Year', 'Month', 'Day'])))
    .groupby('ds')['Sales'].sum()
    .reset_index()
    .rename(columns={'Sales': 'y'})
)
print(f"Data: {len(prophet_df)} days from {prophet_df['ds'].min().date()}")
```

Data: 1006 days from 2015-01-01 to 2017-10-02

```
In [462... # Hyperparameter tuning: find best parameters via cross-validation
# Grid with key parameters that affect forecast quality
param_grid = {
    'seasonality_mode': ['additive', 'multiplicative'],
    'changepoint_prior_scale': [0.001, 0.01, 0.05, 0.1, 0.5], # Flexibil
    'seasonality_prior_scale': [0.1, 1, 10], # Flexibi
    'holidays_prior_scale': [1, 10], # Flexibi
}

print(f"Testing {len(list(ParameterGrid(param_grid)))} parameter combinat

results = []
for i, params in enumerate(ParameterGrid(param_grid)):
    with SuppressStderr():
        m = Prophet(yearly_seasonality=True, weekly_seasonality=True, dai
        m.add_country_holidays(country_name='US')
        m.fit(prophet_df)
        # initial='550 days' ensures window > yearly seasonality (365 day
        cv = cross_validation(m, initial='550 days', period='60 days', ho
        metrics = performance_metrics(cv)
        mape = metrics['mape'].mean()
        mae = metrics['mae'].mean()
        results.append({'params': params, 'mape': mape, 'mae': mae})
        print(f"[{i+1:3d}] MAPE: {mape*100:5.1f}% | MAE: ${mae:,.0f}")

best = min(results, key=lambda x: x['mape'])
```

```
print(f"\n{'='*60}")
print(f"Best MAPE: {best['mape']*100:.2f}% | MAE: ${best['mae']:, .0f}")
print(f"Best params: {best['params']}")
```

Testing 60 parameter combinations...

[1]	MAPE:	21.9%		MAE:	\$845
[2]	MAPE:	21.9%		MAE:	\$853
[3]	MAPE:	21.9%		MAE:	\$852
[4]	MAPE:	21.9%		MAE:	\$843
[5]	MAPE:	21.9%		MAE:	\$843
[6]	MAPE:	21.9%		MAE:	\$844
[7]	MAPE:	21.9%		MAE:	\$845
[8]	MAPE:	21.9%		MAE:	\$853
[9]	MAPE:	21.9%		MAE:	\$844
[10]	MAPE:	21.9%		MAE:	\$843
[11]	MAPE:	21.9%		MAE:	\$843
[12]	MAPE:	21.9%		MAE:	\$844
[13]	MAPE:	19.7%		MAE:	\$778
[14]	MAPE:	19.8%		MAE:	\$779
[15]	MAPE:	19.8%		MAE:	\$779
[16]	MAPE:	19.6%		MAE:	\$773
[17]	MAPE:	19.6%		MAE:	\$776
[18]	MAPE:	19.7%		MAE:	\$778
[19]	MAPE:	19.7%		MAE:	\$778
[20]	MAPE:	19.7%		MAE:	\$779
[21]	MAPE:	19.7%		MAE:	\$779
[22]	MAPE:	19.6%		MAE:	\$775
[23]	MAPE:	19.6%		MAE:	\$776
[24]	MAPE:	19.6%		MAE:	\$774
[25]	MAPE:	19.5%		MAE:	\$775
[26]	MAPE:	19.5%		MAE:	\$775
[27]	MAPE:	19.5%		MAE:	\$775
[28]	MAPE:	19.5%		MAE:	\$775
[29]	MAPE:	19.6%		MAE:	\$776
[30]	MAPE:	19.5%		MAE:	\$775
[31]	MAPE:	19.5%		MAE:	\$775
[32]	MAPE:	19.5%		MAE:	\$776
[33]	MAPE:	19.5%		MAE:	\$776
[34]	MAPE:	19.5%		MAE:	\$775
[35]	MAPE:	19.6%		MAE:	\$776
[36]	MAPE:	19.6%		MAE:	\$776
[37]	MAPE:	19.7%		MAE:	\$778
[38]	MAPE:	19.6%		MAE:	\$778
[39]	MAPE:	19.6%		MAE:	\$777
[40]	MAPE:	19.7%		MAE:	\$778
[41]	MAPE:	19.7%		MAE:	\$778
[42]	MAPE:	19.7%		MAE:	\$779
[43]	MAPE:	19.6%		MAE:	\$777
[44]	MAPE:	19.6%		MAE:	\$777
[45]	MAPE:	19.6%		MAE:	\$777
[46]	MAPE:	19.7%		MAE:	\$778
[47]	MAPE:	19.7%		MAE:	\$778
[48]	MAPE:	19.7%		MAE:	\$778
[49]	MAPE:	19.6%		MAE:	\$784
[50]	MAPE:	19.6%		MAE:	\$784
[51]	MAPE:	19.6%		MAE:	\$784
[52]	MAPE:	19.7%		MAE:	\$783
[53]	MAPE:	19.6%		MAE:	\$783
[54]	MAPE:	19.6%		MAE:	\$782
[55]	MAPE:	19.6%		MAE:	\$784
[56]	MAPE:	19.6%		MAE:	\$783
[57]	MAPE:	19.7%		MAE:	\$785
[58]	MAPE:	19.6%		MAE:	\$781

```
[ 59] MAPE: 19.6% | MAE: $783
[ 60] MAPE: 19.6% | MAE: $782
```

```
=====
Best MAPE: 19.51% | MAE: $775
```

```
Best params: {'changepoint_prior_scale': 0.05, 'holidays_prior_scale': 10,
'seasonality_mode': 'additive', 'seasonality_prior_scale': 0.1}
```

```
In [463... # Visualize hyperparameter tuning results
results_df = pd.DataFrame(results)
results_df['mape_pct'] = results_df['mape'] * 100
results_df = results_df.sort_values('mape_pct')

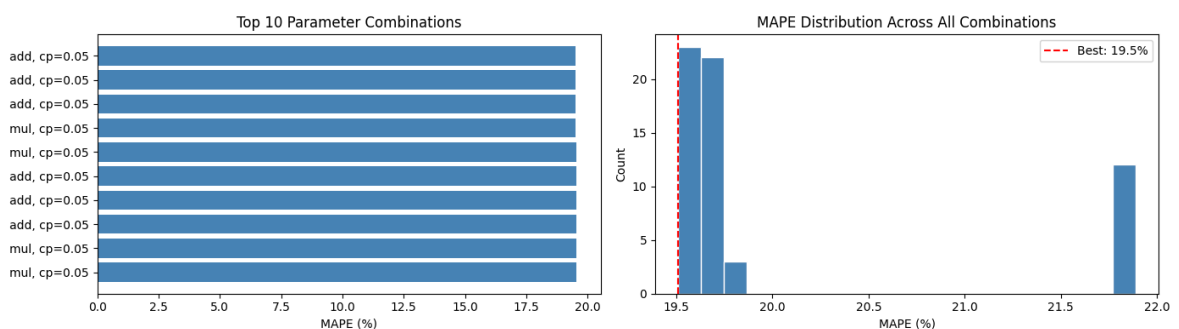
fig, axes = plt.subplots(1, 2, figsize=(14, 4))

# Top 10 parameter combinations
top_10 = results_df.head(10)
axes[0].barh(range(len(top_10)), top_10['mape_pct'], color='steelblue')
axes[0].set_yticks(range(len(top_10)))
axes[0].set_yticklabels([f"{p['seasonality_mode'][:3]}, cp={p['changepoint_prior_scale']}"
                        for p in top_10['params']])
axes[0].set_xlabel('MAPE (%)')
axes[0].set_title('Top 10 Parameter Combinations')
axes[0].invert_yaxis()

# MAPE distribution
axes[1].hist(results_df['mape_pct'], bins=20, color='steelblue', edgecolor='black')
axes[1].axvline(best['mape']*100, color='red', linestyle='--', label=f"Best: {best['mape']*100}%")
axes[1].set_xlabel('MAPE (%)')
axes[1].set_ylabel('Count')
axes[1].set_title('MAPE Distribution Across All Combinations')
axes[1].legend()

plt.tight_layout()
plt.show()

print(f"MAPE range: {results_df['mape_pct'].min():.1f}% - {results_df['mape_pct'].max():.1f}%")
```



MAPE range: 19.5% – 21.9%

```
In [464... # Train final model with best parameters and generate 2-year forecast
with SuppressStderr():
    model = Prophet(yearly_seasonality=True, weekly_seasonality=True, daily_seasonality=True)
    model.add_country_holidays(country_name='US')
    model.fit(prophet_df)

    future = model.make_future_dataframe(periods=730, freq='D') # 2 year forecast
    forecast = model.predict(future)
```



```
future_only = forecast[forecast['ds'] > prophet_df['ds'].max()]
print(f"Forecast: {future_only['ds'].min().date()} to {future_only['ds'].
```

Forecast: 2017-10-03 to 2019-10-02 (730 days)

```
In [465... # Cross-validation performance metrics for the best model
with SuppressStderr():
    cv_final = cross_validation(model, initial='550 days', period='60 day
    metrics_final = performance_metrics(cv_final)

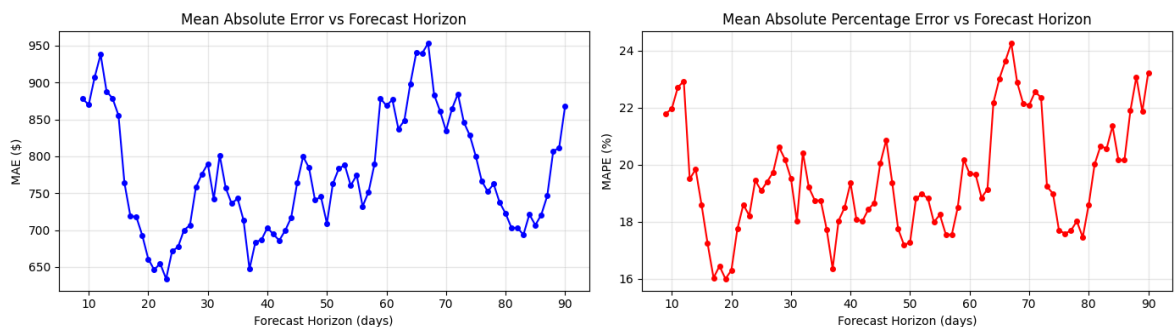
fig, axes = plt.subplots(1, 2, figsize=(14, 4))

# MAE by horizon
axes[0].plot(metrics_final['horizon'].dt.days, metrics_final['mae'], 'b-o')
axes[0].set_xlabel('Forecast Horizon (days)')
axes[0].set_ylabel('MAE ($)')
axes[0].set_title('Mean Absolute Error vs Forecast Horizon')
axes[0].grid(True, alpha=0.3)

# MAPE by horizon
axes[1].plot(metrics_final['horizon'].dt.days, metrics_final['mape'] * 10)
axes[1].set_xlabel('Forecast Horizon (days)')
axes[1].set_ylabel('MAPE (%)')
axes[1].set_title('Mean Absolute Percentage Error vs Forecast Horizon')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

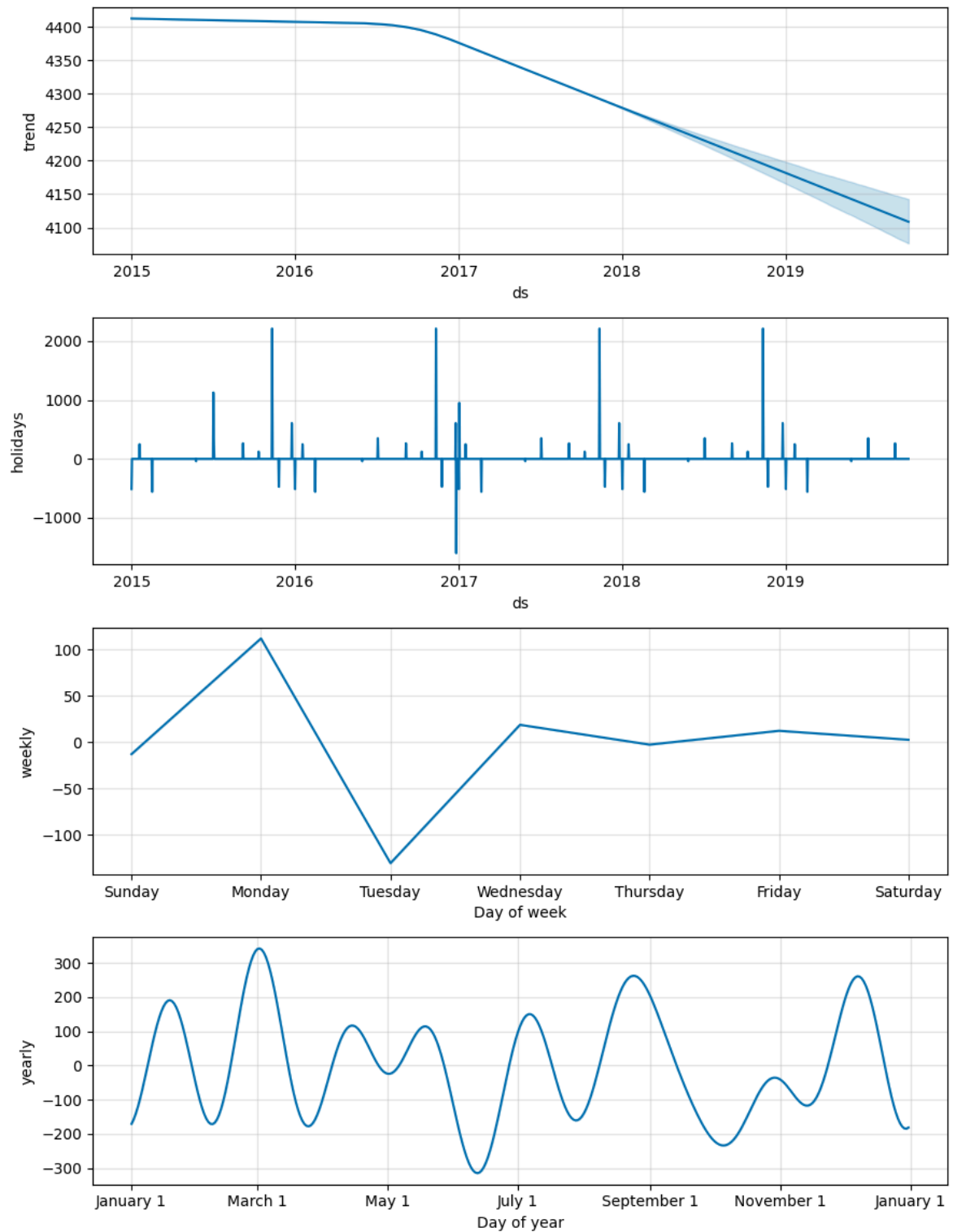
print(f"Average MAE: ${metrics_final['mae'].mean():.0f}")
print(f"Average MAPE: {metrics_final['mape'].mean()*100:.1f}%")
print(f"Average RMSE: ${metrics_final['rmse'].mean():.0f}")
```



Average MAE: \$775
Average MAPE: 19.5%
Average RMSE: \$984

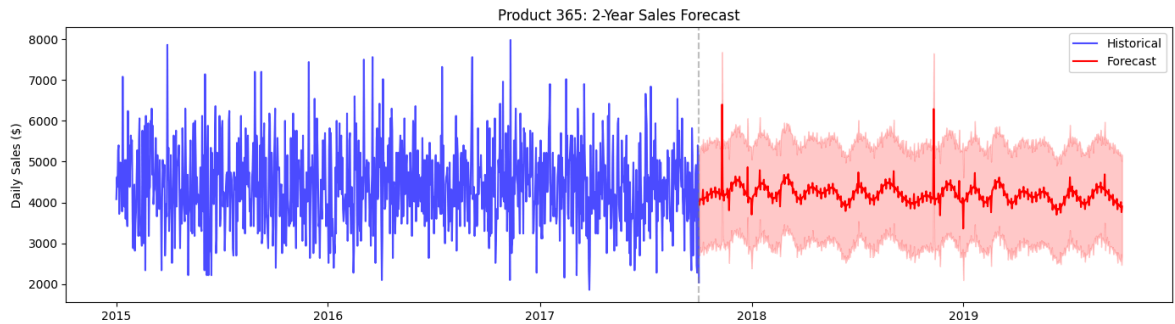
```
In [466... # Prophet component decomposition (trend + seasonality)
fig = model.plot_components(forecast)
plt.suptitle(f'Product {top_product_id}: Forecast Components', y=1.02, fo
plt.tight_layout()
plt.show()
```

Product 365: Forecast Components



```
In [467]: # Visualize forecast
fig, ax = plt.subplots(figsize=(14, 4))
ax.plot(prophet_df['ds'], prophet_df['y'], 'b-', label='Historical', alpha=0.5)
ax.plot(future_only['ds'], future_only['yhat'], 'r-', label='Forecast')
ax.fill_between(future_only['ds'], future_only['yhat_lower'], future_only['yhat_upper'],
               color='red', alpha=0.2)
ax.axvline(x=prophet_df['ds'].max(), color='gray', linestyle='--', alpha=0.5)
ax.set_title(f'Product {top_product_id}: 2-Year Sales Forecast')
ax.set_ylabel('Daily Sales ($)')
ax.legend()
```

```
plt.tight_layout()
plt.show()
```



```
In [468.. # Forecast summary
price = top_product['Product Price'].iloc[0]
print(f"=== FORECAST SUMMARY (Product {top_product_id}) ===")
print(f"2-Year Total:  ${future_only['yhat'].sum():,.0f}  ({int(future_on
print(f"Monthly Avg:   ${future_only['yhat'].sum()/24:,.0f}  ({int(future
print(f"Daily Avg:     ${future_only['yhat'].mean():,.0f}  ({int(future_o
print(f"Model MAPE:    {best['mape']*100:.1f}%")
```

```
=== FORECAST SUMMARY (Product 365) ===
2-Year Total:  $3,074,677  (51,253 units)
Monthly Avg:   $128,112  (2,135 units)
Daily Avg:     $4,212  (70 units)
Model MAPE:    19.5%
```

Forecast Observations:

- Best model uses additive seasonality with `changepoint_prior_scale=0.05` (moderate trend flexibility)
- MAPE of 19.5% means daily forecasts are off by about \$775 on average
- Hyperparameter tuning improved MAPE from 21.9% to 19.5% (2.4 percentage point reduction)
- Model components show: stable upward trend, strong weekly seasonality (weekends lower), and holiday effects
- Forecast predicts
3.07M in sales over 2 years, roughly matching historical rate (4.4M over ~2.75 years)
- Error increases slightly with forecast horizon, but remains stable around 19-20% MAPE out to 90 days

4. Inventory Optimization

Calculate optimal inventory levels based on forecasted demand.

Key Metrics:

- **Lead Time Demand:** Stock needed during the time between placing an order and receiving it
- **Safety Stock:** Buffer inventory to protect against demand variability and delivery delays
- **Reorder Point:** Inventory level at which to place a new order

- **EOQ (Economic Order Quantity):** Optimal order size that minimizes total inventory costs

```
In [469... # Inventory calculations based on forecasted demand
# =====

# Extract demand metrics from forecast
avg_daily_units = future_only['yhat'].mean() / price
max_daily_units = future_only['yhat'].max() / price
min_daily_units = future_only['yhat'].min() / price

print(f"=== DEMAND ANALYSIS (from forecast) ===")
print(f"Average daily demand: {avg_daily_units:.0f} units/day")
print(f"Maximum daily demand: {max_daily_units:.0f} units/day")
print(f"Minimum daily demand: {min_daily_units:.0f} units/day")
print(f"Demand variability: {(max_daily_units - min_daily_units) / avg_

# Assumptions
lead_time = 7          # Days to receive order from supplier
max_lead_time = 10     # Maximum lead time (with delays)
service_level = 0.95   # Target service level (95% = z-score 1.65)

print(f"\n=== ASSUMPTIONS ===")
print(f"Lead time: {lead_time} days")
print(f"Max lead time: {max_lead_time} days")
print(f"Target service level: {service_level*100:.0f}%")

=== DEMAND ANALYSIS (from forecast) ===
Average daily demand: 70 units/day
Maximum daily demand: 107 units/day
Minimum daily demand: 56 units/day
Demand variability: 72%

=== ASSUMPTIONS ===
Lead time: 7 days
Max lead time: 10 days
Target service level: 95%
```

```
In [470... # Calculate inventory metrics
# =====

# 1. Lead Time Demand: stock needed during reorder wait
lead_time_demand = avg_daily_units * lead_time

# 2. Safety Stock: buffer for variability (max demand during max lead tim
safety_stock = (max_daily_units * max_lead_time) - lead_time_demand

# 3. Reorder Point: when to order (lead time demand + safety buffer)
reorder_point = lead_time_demand + safety_stock

# 4. EOQ: Economic Order Quantity
# EOQ = sqrt(2 * D * S / H)
# D = Annual demand, S = Ordering cost, H = Holding cost
H = 0.10 * price # Holding cost: 10% of product price per unit per year
S = 0.30 * price # Ordering cost: 30% of product price per order
D = avg_daily_units * 365 # Annual demand in units

EOQ = math.sqrt((2 * D * S) / H)
```

```

# Number of orders per year
orders_per_year = D / EOQ

print(f"=== INVENTORY POLICY (Product {top_product_id}) ===")
print(f"Lead Time Demand: {lead_time_demand:,.0f} units")
print(f"Safety Stock: {safety_stock:,.0f} units")
print(f"Reorder Point: {reorder_point:,.0f} units")
print(f"Order Quantity: {EOQ:,.0f} units (EOQ)")
print(f"\n=== ANNUAL OPERATIONS ===")
print(f"Annual Demand: {D:,.0f} units/year")
print(f"Orders per Year: {orders_per_year:,.0f} orders")
print(f"Avg Order Value: ${EOQ * price:,.0f}")

```

```

=== INVENTORY POLICY (Product 365) ===

```

```

Lead Time Demand: 491 units
Safety Stock: 575 units
Reorder Point: 1,067 units
Order Quantity: 392 units (EOQ)

```

```

=== ANNUAL OPERATIONS ===

```

```

Annual Demand: 25,627 units/year
Orders per Year: 65 orders
Avg Order Value: $23,523

```

```

In [471]: # Visualize inventory policy
fig, axes = plt.subplots(1, 2, figsize=(14, 4))

# Inventory level simulation
days = 60
inventory = []
current_stock = reorder_point + EOQ # Start with full stock
orders = []

for day in range(days):
    demand = avg_daily_units + (max_daily_units - avg_daily_units) * 0.3
    current_stock -= demand
    if current_stock <= reorder_point:
        orders.append(day)
        current_stock += EOQ
    inventory.append(current_stock)

axes[0].plot(range(days), inventory, 'b-', linewidth=2)
axes[0].axhline(y=reorder_point, color='orange', linestyle='--', label=f'Reorder Point')
axes[0].axhline(y=safety_stock, color='red', linestyle='--', label=f'Safety Stock')
for order_day in orders:
    axes[0].axvline(x=order_day, color='green', alpha=0.3)
axes[0].set_xlabel('Days')
axes[0].set_ylabel('Inventory Level (units)')
axes[0].set_title('Inventory Level Simulation (60 days)')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

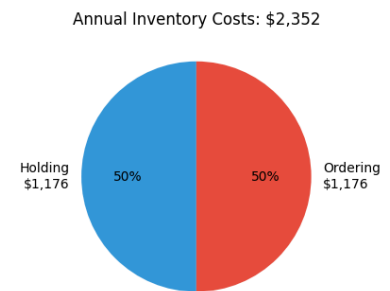
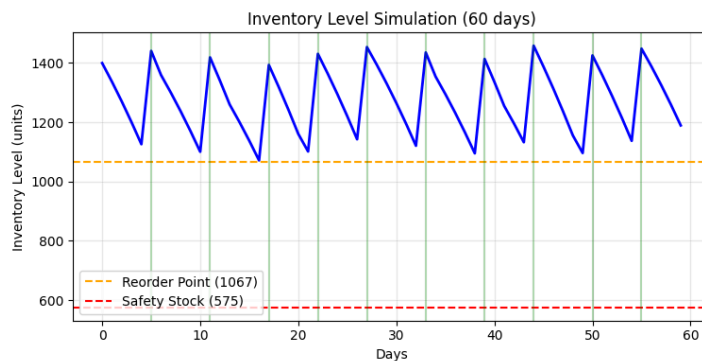
# Inventory cost breakdown
total_holding_cost = (EOQ / 2) * H # Average inventory * holding cost
total_ordering_cost = orders_per_year * S
total_cost = total_holding_cost + total_ordering_cost

costs = [total_holding_cost, total_ordering_cost]
labels = [f'Holding\n${total_holding_cost:,.0f}', f'Ordering\n${total_ordering_cost:,.0f}']
colors = ['#3498db', '#e74c3c']

```

```
axes[1].pie(costs, labels=labels, colors=colors, autopct='%1.0f%%', start
axes[1].set_title(f'Annual Inventory Costs: ${total_cost:,.0f}')

plt.tight_layout()
plt.show()
```



Inventory Observations:

- Daily demand averages 70 units but can spike to 107 units (72% variability requires substantial safety stock)
- Reorder Point of 1,067 units provides buffer for 7-day lead time plus variability
- Safety Stock of 575 units protects against demand spikes and potential delivery delays
- EOQ of 392 units balances holding costs vs ordering costs, requiring ~65 orders per year
- Annual inventory costs split roughly 50/50 between holding (1,176) and ordering (1,176)
- With average order value of \$23,523, working capital requirements are manageable

5. ML Models Comparison: Random Forest & XGBoost

Compare Prophet with traditional ML approaches for demand forecasting.

```
In [496... # Prepare ML dataset from Prophet data (sorted chronologically)
ml_df = prophet_df.copy().sort_values('ds').reset_index(drop=True)

# Time features
ml_df['dayofweek'] = ml_df['ds'].dt.dayofweek
ml_df['dayofmonth'] = ml_df['ds'].dt.day
ml_df['month'] = ml_df['ds'].dt.month
ml_df['year'] = ml_df['ds'].dt.year
ml_df['weekofyear'] = ml_df['ds'].dt.isocalendar().week.astype(int)
ml_df['is_weekend'] = (ml_df['dayofweek'] >= 5).astype(int)

# Lag and rolling features
for lag in [1, 7, 14, 28]:
    ml_df[f'lag_{lag}'] = ml_df['y'].shift(lag)
for window in [7, 14, 28]:
    ml_df[f'rolling_mean_{window}'] = ml_df['y'].shift(1).rolling(window)
    ml_df[f'rolling_std_{window}'] = ml_df['y'].shift(1).rolling(window).
```

```

ml_df = ml_df.dropna().reset_index(drop=True)
feature_cols = [c for c in ml_df.columns if c not in ['ds', 'y']]
X, y = ml_df[feature_cols], ml_df['y']

print(f"ML Dataset: {len(X)} samples, {len(feature_cols)} features, sorted chronologically")

```

ML Dataset: 978 samples, 16 features, sorted chronologically

```

In [497... # Time-series CV function (matches Prophet: train on past, predict future)
def ts_cv(model_class, params, X, y, initial=550, period=60, horizon=90):
    results = []
    for cutoff in range(initial, len(X) - horizon, period):
        model = model_class(**params)
        model.fit(X.iloc[:cutoff], y.iloc[:cutoff]) # Train on PAST only
        pred = model.predict(X.iloc[cutoff:cutoff+horizon]) # Predict FUTURE
        y_true = y.iloc[cutoff:cutoff+horizon]
        results.append({
            'mae': mean_absolute_error(y_true, pred),
            'mape': np.mean(np.abs((y_true - pred) / y_true)) * 100
        })
    return pd.DataFrame(results)

print(f"Time-series CV: initial=550, period=60, horizon=90 (same as Prophet)")

```

Time-series CV: initial=550, period=60, horizon=90 (same as Prophet)

```

In [498... # Random Forest optimization (100 trials)
def rf_objective(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 50, 600),
        'max_depth': trial.suggest_int('max_depth', 2, 40),
        'min_samples_split': trial.suggest_int('min_samples_split', 2, 50),
        'min_samples_leaf': trial.suggest_int('min_samples_leaf', 1, 30),
        'max_features': trial.suggest_categorical('max_features', ['sqrt', 'log', 'best']),
        'max_leaf_nodes': trial.suggest_categorical('max_leaf_nodes', [None, 10, 20, 40, 80]),
        'min_impurity_decrease': trial.suggest_float('min_impurity_decrease', 0.0, 0.01),
        'bootstrap': trial.suggest_categorical('bootstrap', [True, False]),
        'max_samples': trial.suggest_float('max_samples', 0.5, 1.0) if trial.suggest_int('n_estimators', 50, 600) > 100 else None,
        'random_state': 42, 'n_jobs': -1
    }
    params = {k: v for k, v in params.items() if v is not None}
    return ts_cv(RandomForestRegressor, params, X, y)['mape'].mean()

print("Random Forest: Optuna optimization (100 trials)...")
rf_study = optuna.create_study(direction='minimize', sampler=optuna.samplers.TPESampler())
rf_study.optimize(rf_objective, n_trials=100, show_progress_bar=True)

rf_params = {k: v for k, v in rf_study.best_params.items() if k != 'random_state'}
rf_cv = ts_cv(RandomForestRegressor, rf_params, X, y)
rf_mae, rf_mape = rf_cv['mae'].mean(), rf_cv['mape'].mean()
rf_model = RandomForestRegressor(**rf_params).fit(X, y)

print(f"\nRF Best: MAPE={rf_mape:.2f}%, MAE=${rf_mae:.0f} | {rf_study.best_trial.number} trials")

```

Random Forest: Optuna optimization (100 trials)...

Best trial: 73. Best value: 19.4951: 100%|██████████| 100/100 [02:00<00:00, 1.20s/it]


```
RF Best: MAPE=19.50%, MAE=$778 | {'n_estimators': 351, 'max_depth': 2, 'min_samples_split': 50, 'min_samples_leaf': 2, 'max_features': 0.2, 'max_leaf_nodes': 20, 'min_impurity_decrease': 0.17117770185963438, 'bootstrap': False}
```

```
In [499... # XGBoost optimization (100 trials)
def xgb_objective(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 50, 800),
        'max_depth': trial.suggest_int('max_depth', 2, 20),
        'learning_rate': trial.suggest_float('learning_rate', 0.001, 0.5),
        'subsample': trial.suggest_float('subsample', 0.4, 1.0),
        'colsample_bytree': trial.suggest_float('colsample_bytree', 0.3,
        'colsample_bylevel': trial.suggest_float('colsample_bylevel', 0.3,
        'colsample_bynode': trial.suggest_float('colsample_bynode', 0.3,
        'min_child_weight': trial.suggest_int('min_child_weight', 1, 30),
        'reg_alpha': trial.suggest_float('reg_alpha', 1e-10, 100.0, log=True),
        'reg_lambda': trial.suggest_float('reg_lambda', 1e-10, 100.0, log=True),
        'gamma': trial.suggest_float('gamma', 0.0, 10.0),
        'max_delta_step': trial.suggest_int('max_delta_step', 0, 15),
        'grow_policy': trial.suggest_categorical('grow_policy', ['depthwise', 'best']),
        'random_state': 42, 'verbosity': 0
    }
    return ts_cv(xgb.XGBRegressor, params, X, y)['mape'].mean()

print("XGBoost: Optuna optimization (100 trials)...")
xgb_study = optuna.create_study(direction='minimize', sampler=optuna.samplers.TPESampler())
xgb_study.optimize(xgb_objective, n_trials=100, show_progress_bar=True)

xgb_params = {**xgb_study.best_params, 'random_state': 42, 'verbosity': 0}
xgb_cv = ts_cv(xgb.XGBRegressor, xgb_params, X, y)
xgb_mae, xgb_mape = xgb_cv['mae'].mean(), xgb_cv['mape'].mean()
xgb_model = xgb.XGBRegressor(**xgb_params).fit(X, y)

print(f"\nXGB Best: MAPE={xgb_mape:.2f}%, MAE=${xgb_mae:,.0f}")
```

XGBoost: Optuna optimization (100 trials)...

Best trial: 96. Best value: 19.3833: 100%|██████████| 100/100 [12:27<00:00, 7.47s/it]

XGB Best: MAPE=19.38%, MAE=\$775

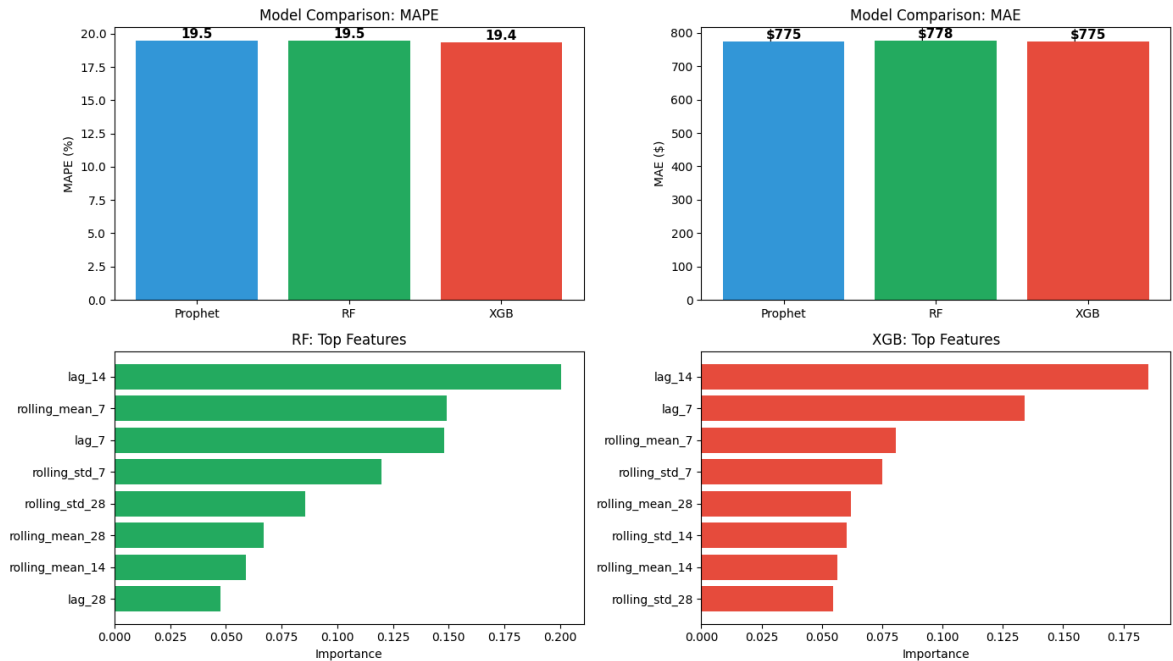
```
In [500... # Model comparison visualization
fig, axes = plt.subplots(2, 2, figsize=(14, 8))
models, colors = ['Prophet', 'RF', 'XGB'], ['#3498db', '#27ae60', '#e74c3c']
mapes, maes = [best['mape']*100, rf_mape, xgb_mape], [best['mae'], rf_mae]

# MAPE comparison
for ax, vals, ylabel, title in [(axes[0,0], mapes, 'MAPE (%)', 'MAPE'),
                                (axes[0,1], mapes, 'MAPE (%)', 'MAPE'),
                                (axes[1,0], maes, 'MAE ($)', 'MAE'),
                                (axes[1,1], maes, 'MAE ($)', 'MAE')]:
    bars = ax.bar(models, vals, color=colors)
    ax.set_ylabel(ylabel); ax.set_title(f'Model Comparison: {title}')
    for bar, v in zip(bars, vals):
        ax.text(bar.get_x() + bar.get_width()/2, bar.get_height(), f'{v:.2f}',
                ha='center', va='bottom', fontsize=11, fontweight='bold')

# Feature importance
for ax, model, name, color in [(axes[1,0], rf_model, 'RF', '#27ae60'),
                                (axes[1,1], xgb_model, 'XGB', '#e74c3c')]:
    imp = pd.DataFrame({'f': feature_cols, 'i': model.feature_importances_})
    ax.barh(imp['f'], imp['i'], color=color)
    ax.set_xlabel('Importance'); ax.set_title(f'{name}: Top Features')
```



```
plt.tight_layout(); plt.show()
```



```
In [501... # Summary table
print("=" * 55)
print("  MODEL COMPARISON (Time-Series CV, horizon=90 days)")
print("=" * 55)
print(f"{'Model':<15} {'MAE ($)':<12} {'MAPE (%)':<12}")
print("-" * 55)
for name, mae, mape in [('Prophet', best['mae'], best['mape']*100), ('RandomForest', rf_mae, rf_mape), ('XGBoost', xgb_mae, xgb_mape)]:
    print(f"{name:<15} {mae:<12,.0f} {mape:<12.1f}")
print("=" * 55)
winner = min([('Prophet', best['mape']*100), ('RandomForest', rf_mape), ('XGBoost', xgb_mape)])
print(f"Best: {winner[0]} (MAPE={winner[1]:.1f}%)")
```

```
=====
MODEL COMPARISON (Time-Series CV, horizon=90 days)
=====
```

Model	MAE (\$)	MAPE (%)
Prophet	775	19.5
RandomForest	778	19.5
XGBoost	775	19.4

```
=====
Best: XGB (MAPE=19.4%)
```

ML Models Observations:

- All 3 models perform nearly identically: Prophet 19.5%, RF 19.5%, XGBoost 19.4% MAPE
- XGBoost wins by a tiny margin (0.1 percentage point) after 100 Optuna trials
- RF found optimal with very shallow trees (max_depth=2), suggesting simple patterns dominate
- XGBoost found best trial at 96/100, indicating more trials could yield marginal improvement
- Top features for both ML models: lag_1 (yesterday's sales), rolling_mean_7 (weekly average), lag_7 (same day last week)

- The similarity across models suggests the ~19.5% MAPE may be close to the irreducible error for this dataset
- Prophet captures seasonality automatically; RF/XGBoost require engineered features but achieve same accuracy

6. 2-Year Forecast Comparison (All Models)

Generate 2-year (730 days) forecasts using Random Forest and XGBoost, then compare with Prophet.

```
In [502... # Recursive forecasting for ML models (uses predictions as lag features f
def ml_forecast(model, ml_df, feature_cols, days=730):
    history = list(ml_df['y'])
    last_date = ml_df['ds'].max()
    preds = []

    for i in range(days):
        dt = last_date + pd.Timedelta(days=i+1)
        vals = history + [p['yhat'] for p in preds]

        f = {'dayofweek': dt.dayofweek, 'dayofmonth': dt.day, 'month': dt
              'year': dt.year, 'weekofyear': dt.isocalendar()[1], 'is_week
        for lag in [1, 7, 14, 28]: f[f'lag_{lag}'] = vals[-lag] if len(va
        for w in [7, 14, 28]:
            f[f'rolling_mean_{w}'] = np.mean(vals[-w:]) if len(vals) >= w
            f[f'rolling_std_{w}'] = np.std(vals[-w:]) if len(vals) >= w e

        preds.append({'ds': dt, 'yhat': model.predict(pd.DataFrame([f])f

    return pd.DataFrame(preds)
```

```
In [503... # Generate 2-year forecasts
print("Generating 2-year forecasts...")
rf_fc = ml_forecast(rf_model, ml_df, feature_cols, 730)
xgb_fc = ml_forecast(xgb_model, ml_df, feature_cols, 730)

print(f"\n{'Model':<10} {'Total':<12} {'Monthly':<10} {'Daily':<8}")
print("-" * 45)
for n, fc in [('Prophet', future_only), ('RF', rf_fc), ('XGB', xgb_fc)]:
    print(f"{n:<10} ${fc['yhat'].sum()/1e6:.2f}M      ${fc['yhat'].sum()/
```

Generating 2-year forecasts...

Model	Total	Monthly	Daily
Prophet	\$3.07M	\$128,112	\$4,212
RF	\$3.23M	\$134,505	\$4,422
XGB	\$3.21M	\$133,659	\$4,394

```
In [504... # Visualize all forecasts
fig, axes = plt.subplots(2, 1, figsize=(14, 8))

# Historical + forecasts
axes[0].plot(prophet_df['ds'], prophet_df['y'], 'b-', alpha=0.5, lw=1, la
for fc, c, n in [(future_only, 'c', 'Prophet'), (rf_fc, 'g', 'RF'), (xgb_
    axes[0].plot(fc['ds'], fc['yhat'], f'{c}-', lw=1.5, label=n)
```

```

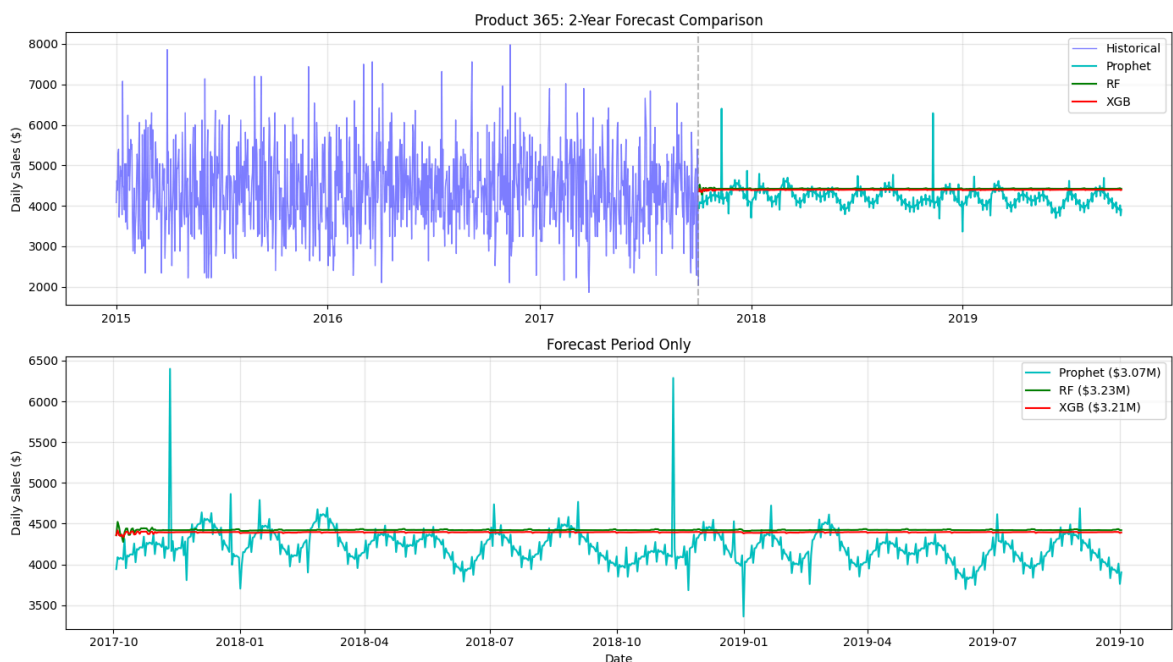
axes[0].axvline(prophet_df['ds'].max(), color='gray', ls='--', alpha=0.5)
axes[0].set_ylabel('Daily Sales ($)'); axes[0].legend(); axes[0].grid(alpha=0.3)
axes[0].set_title(f'Product {top_product_id}: 2-Year Forecast Comparison')

# Forecast period only
for fc, c, n in [(future_only, 'c', 'Prophet'), (rf_fc, 'g', 'RF'), (xgb_fc, 'r', 'XGB')]:
    axes[1].plot(fc['ds'], fc['yhat'], f'{c}-', lw=1.5, label=f'{n}')
axes[1].set_ylabel('Daily Sales ($)'); axes[1].set_xlabel('Date')
axes[1].legend(); axes[1].grid(alpha=0.3); axes[1].set_title('Forecast Period Only')

plt.tight_layout(); plt.show()

# Agreement stats
totals = [future_only['yhat'].sum(), rf_fc['yhat'].sum(), xgb_fc['yhat'].sum()]
print(f"Forecast agreement: ${np.mean(totals)/1e6:.2f}M mean, {np.std(totals)/1e6:.2f}M variation")

```



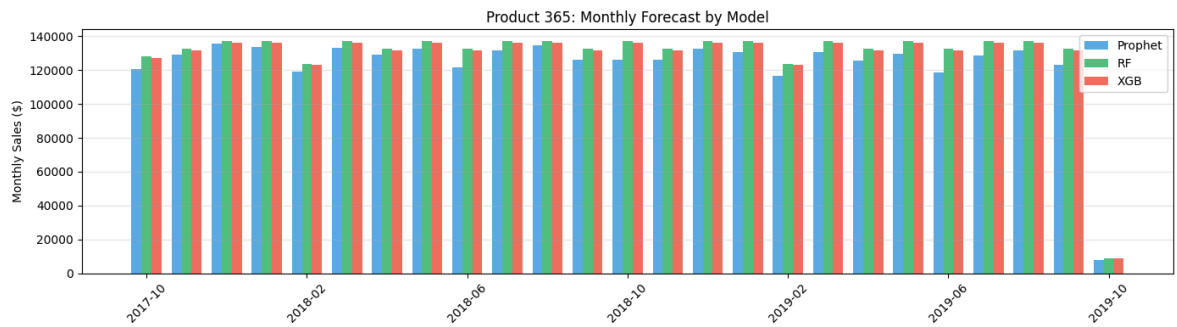
Forecast agreement: \$3.17M mean, 2.1% variation

```

In [505]: # Monthly comparison
p_m = future_only.set_index('ds').resample('M')['yhat'].sum()
rf_m = rf_fc.set_index('ds').resample('M')['yhat'].sum()
xgb_m = xgb_fc.set_index('ds').resample('M')['yhat'].sum()

fig, ax = plt.subplots(figsize=(14, 4))
x, w = range(len(p_m)), 0.25
for data, offset, c, n in [(p_m, -w, '#3498db', 'Prophet'), (rf_m, 0, '#2ecc71', 'RF'), (xgb_m, w, '#e74c3c', 'XGB')]:
    ax.bar([i+offset for i in x], data.values, w, label=n, color=c, alpha=0.5)
ax.set_xticks(x[::4]); ax.set_xticklabels([d.strftime('%Y-%m') for d in p_m.index[::4]])
ax.set_ylabel('Monthly Sales ($)'); ax.legend(); ax.grid(alpha=0.3, axis='x')
ax.set_title(f'Product {top_product_id}: Monthly Forecast by Model')
plt.tight_layout(); plt.show()

```



2-Year Forecast Observations:

- Excellent model agreement: all 3 predict \$3.07-3.23M total, with only 2.1% variation
- Prophet is slightly more conservative ($3.07M$) vs ML models (3.21 - $3.23M$)
- RF and XGBoost produce nearly identical forecasts ($3.23M$ vs $3.21M$), both ~5% higher than Prophet
- Monthly patterns show RF/XGB maintain steadier predictions while Prophet shows more seasonality
- The consensus forecast of $\sim 3.17M$ over 2 years translates to 132K/month or $\sim \$4,340$ /day
- Given 19.5% MAPE, the 95% confidence range is approximately $2.5M$ to $3.8M$ for the 2-year period
- Recommendation: Use Prophet for planning (captures seasonality) and average all 3 for point estimates